

CMSC 471: Introduction to Artificial Intelligence

Name: Mustabshirah Musarrat

Section: 3

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining cost it lower bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Answer:

Steps: For the steps cost, every legal letter replacement costs 1. The heuristic counts the number of positions where the current word differ from the goal word.

Here $h(n)$ = number of positions i where $\text{current}[i]$ is different from $\text{goal}[i]$

Pseudocode:

```
count = 0;
```

```
for each position i:
```

```
    if  $\text{current}[i] \neq \text{goal}[i]$ :
```

```
        count = count + 1
```

```
return count
```

Every position that is different from the goal must be changed. Since one action changes exactly one letter, at least one action is required for each mismatched position, which implies each action costs must be exactly 1. This means that if there are k mismatched positions, at least k more actions are required. So, $h(n) = k$ is a lower bound on the remaining cost.

This is admissible because every position that is different from the goal must be changed. One action change only one letter and every action costs exactly 1.

Therefore, $h(n) \leq \text{actual remaining cost}$. The heuristic cannot overestimate the cost. At the goal, there are no mismatched positions, so $h(\text{goal}) = 0$.

Scrabble: For the scrabble cost, the cost of an action is the Scrabble value of the letter begin entered. The heuristic looks at every position that does not match the goal and adds the Scrabble value of the goal letter at the position.

$h(n) = \text{sum of Scrabble values of the goal letters at all mismatched positions}$

Pseudocode:

total = 0;

for each position i:

 if current[i] != goal[i]:

 total = total + ScrabbleValue(goal[i])

return total

Now for every mismatched position must contain the corresponding goal letter. To make a mismatched position correct, an action must enter the goal letter for that position. For example, if a mismatched position needs the letter c that required change has a scrabble cost 3. So, the scrabble value of each required goal letter is an unavoidable part of the remaining cost. Therefore, the sum of these values is a lower bound on the total remaining cost.

The heuristic assumes that each mismatched position only needs the costs of entering its final goal letter. Also the actual solution may be changed as the intermediate words need to be legal based on the dictionary words. So, the actual solution can cost same or more than the lower bound, but it cannot cost less than the scrabble values of the required goal letter.

Therefore, $h(n) \leq \text{actual remaining scrabble cost}$.

The heuristic is also 0 at the goal because there are no mismatched positions.

Frequency: For frequency cost, the actual frequency cost of entering a word w is $1 + \text{rarity}(w)$. The program implements this in path_cost() as:

rarity= dictionary[state2]

return c+1+rarity

Here, state2 is the new word being entered. Therefore, the actual cost of entering a new word is exactly $1 + \text{rarity}(\text{new word})$.

For example, if three positions are different, at least three letter changing actions are required. So, $h(n) = \text{number of positions } i \text{ where } \text{current}[i] \neq \text{goal}[i]$.

Pseudocode:

```
count = 0;
```

```
for each position i:
```

```
    if state[i] != goal[i]:
```

```
        count = count+ 1
```

```
return count
```

If k position differ from the goal, at least k actions necessary because each action changes only one position. For the frequency cost, every action costs $1 + \text{rarity}(w)$. Since $\text{rarity}(w)$ is nonnegative, and every action costs at least 1. Therefore, at least k cost is unavoidable.

The heuristic returns k , the minimum number of actions that are required. Since every frequency action costs at least 1, the actual remaining frequency cost must be at least k . Therefore, the heuristic cannot overestimate the true optimal remaining cost.

Here the heuristic is admissible because every mismatched position requires at least one action, and every frequency action costs at least 1: $1 + \text{rarity}(w) \geq 1$.

Thus, $h(n) \leq \text{actual remaining frequency cost}$

At the goal, there are no mismatched positions, so the heuristic is 0.

Question 2: Does the heuristic actually help? [10 points]

Run both uniform-cost search and A* on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.930%

WHY → ASK	Scrabble	9.00	765	9.00	13	98.301%
WHY → ASK	Frequency	38.86	808	38.86	755	6.559%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.132%
BEAR → DUCK	Scrabble	11.00	2981	11.00	6	99.799%
BEAR → DUCK	Frequency	38.24	1084	38.24	720	33.579%

Calculate the percentage reduction as:

$$100 * (\text{UCS expansions} - \text{A}^* \text{ expansions}) / \text{UCS expansions}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Answer: Yes. UCS and A* returned the same total cost for all six experiments. The costs were 5.00, 9.00, and 38.86 for WHY → ASK under steps, Scrabble, and frequency, respectively. For BEAR → DUCK, the costs were 4.00, 11.00, and 38.24. Therefore, there was no discrepancy between the optimal costs returned by UCS and A*.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

Answer: For WHY → ASK, the largest reduction was under the Scrabble cost measure, with a 98.301% reduction in expanded nodes.

For BEAR → DUCK, the largest reduction was also under the Scrabble cost measure, with a 99.799% reduction.

3. For each word pair, under which cost measure was the heuristic least informative?

Answer: For WHY → ASK, the frequency heuristic was the least informative because it produced the smallest reduction, 6.559%.

For BEAR → DUCK, the frequency heuristic was also the least informative because it produced the smallest reduction, 33.579%.

4. Explain the differences using the definitions of your heuristics, not runtime alone. Different optimal paths are acceptable when their total costs are equal.

Answer: The steps heuristic counts the number of positions that differ between the current word and the goal word. Since every step costs 1, every mismatched position requires at least one action. Therefore, the mismatch count provides a useful lower bound for the steps cost.

The Scrabble heuristic considers the scrabble value of the goal letter at each mismatched position. Therefore, it considers both the number of remaining mismatches and the scrabble cost of inserting the required letters. This makes it more closely related to the scrabble cost measure.

The frequency heuristic in this project counts the number of mismatched positions, which is the same basic estimate used by the steps heuristic. But, frequency costs depend on the rarity of the resulting word, because each action costs $1 + \text{rarity}(\text{word})$. The frequency heuristic does not consider these rarity values. As a result, it provides a much weaker estimate of the actual remaining frequency cost for these two experiments, which is reflected by the smaller reductions in nodes expanded.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal. For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from $W1$ to $W2$ must always equal the optimal cost from $W2$ to $W1$. For each cost measure:

1. State yes or no.

Answer: Steps: Yes,

Scrabble: No and

Frequency: No

2. Justify your conclusion.

Answer: Steps: For the steps cost, every legal action costs exactly 1. If a path from $W1$ to $W2$ has k actions, reversing the same path from $W2$ to $W1$ also k actions. Therefore, the cost of the reversed path is the same. Since every path can be reversed without changing its numbers of actions, the optimal cost must also be the same in both directions.

Scrabble: Although every action is reversible, the cost of an action depends on the letter that is inserted. When an action is reversed, the inserted letter is different, so its scrabble value can also be different. Therefore, reversing a path does not necessarily preserve its cost, and the optimal costs in the two directions do not have to be equal.

Frequency: Frequency cost depends on the resulting word. An action that enters one word has cost $1 + \text{rarity}(\text{word})$, while the reverse action enters a different word and can have a different rarity value. So, reversing a path does not necessarily preserve its cost, which implies that the optimal costs in the two directions do not have to be equal.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary.

Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Answer: In the reverse direction, $\text{cog} \rightarrow \text{dog}$ the inserted letter is d, whose Scrabble value is 2. Therefore, this action costs 2. But if the direction is $\text{dog} \rightarrow \text{cog}$, the inserted letter c, whose Scrabble value is 3. So, the action cost is 3.

Thus, the same reversible action has different costs in the two directions.

For frequency, consider the same legal words, $\text{dog} \rightarrow \text{cog}$. The cost is: $1 + \text{rarity}(\text{cog})$, while the reverse: $\text{cog} \rightarrow \text{dog}$ has cost: $1 + \text{rarity}(\text{dog})$.

Because the rarity values of cog and dog are not the same, these two action costs can be different. Therefore, frequency cost is not necessarily symmetric.

Reversibility means that the same legal action can be performed in the opposite direction. Cost symmetry is a separate property. Steps have both reversibility and symmetric costs, while Scrabble and frequency have reversible actions but also direction dependent costs.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as

h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

Answer: I used GPT-5.6 Luna through Chat GPT as the generative AI system. The exact prompt I gave the model: "In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost."

The proposed heuristic is: $h(n) = 0$ if n is the g . $h(n) = h_0(n) + \text{rarity}(g)$ if $n \neq g$, where, h_0 is the number of positions in which the current state n differs from the goal word g and $\text{rarity}(g)$ is the rarity value of the goal word in the supplied dictionary.

2. A precise formula or pseudocode for the proposed heuristic.

Answer: If the current state is the goal, then $h(n) = 0$. Or else $h(n) = h_0(n) + \text{rarity}(g)$

where $h_0(n)$ is the number of positions where n and g have different letters.

Pseudocode:

If state == goal:

 return 0

count = 0

for each position i:

 if state[i] != goal[i]:

 count = count + 1

return count + dictionary[goal]

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

Answer: The proposed heuristic is $h(n) = h_0(n) + \text{rarity}(g)$ for non goal states, where

$h_0(n)$ is the number of positions in the current word that differ from the goal.

Let $k = h_0(n)$. Since each action changes only one letter, at least k actions are required to change all mismatched positions into the goal word. For frequency cost, every action costs $1 + \text{rarity}(w)$ where $\text{rarity}(w) \geq 0$. Therefore, every action cost at least 1.

Also, the final action must enter the goal word. Therefore, the final action costs, $1 + \text{rarity}(\text{goal})$.

The other $k - 1$ required actions cost at least 1 each. Thus, any path from state n to the goal has a cost of at least

$$k - 1 + (1 + \text{rarity}(\text{goal})) = k + \text{rarity}(\text{goal}).$$

Since $k = h_0(n)$, $h(n) = h_0(n) + \text{rarity}(\text{goal})$.

So, the proposed heuristic never overestimates the true optimal remaining cost, so it is admissible.

The heuristic is also at least as large as h_0 because $\text{rarity}(g)$ is nonnegative, $h(n) = h_0(n) + \text{rarity}(g) \geq h_0(n)$. If $\text{rarity}(g)$ is greater than 0, then the proposed heuristic is strictly larger than h_0 for every nongoa state.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

Answer: The proposed heuristic is valid as long as the goal word has a positive rarity value for the particular problem being tested. If $\text{rarity}(g) = 0$, then $h(n) = h_0(n)$.

In that case, the heuristic is still admissible, but it does not satisfy the requirement that it be strictly larger than h_0 for at least one state.

Therefore, the goal rarity should be checked for each problem. If the goal rarity is positive, the proposed heuristic satisfies both requirements, it is admissible and strictly larger than h_0 for non-goal states.

The heuristic is admissibility. The only possible issue is the strict improvement requirement when the goal rarity is 0.

For the two frequency cost problems from Question 2, the goal words is "ASK" for WHY → ASK and "DUCK" for BEAR → DUCK. The rarity values of "ASK" and "DUCK" should be checked in the supplied dictionary to verify that the proposed heuristic is strictly larger than h_0 for these experiments.

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Answer: The two frequency cost problems from Question 2 are:

WHY → ASK

BEAR → DUCK

Experimental results:

To test the proposed heuristic, I temporarily changed the frequency part of $h()$ function in `dc.py` from the original heuristic to the proposed heuristic:

```
elif self.cost == 'frequency':  
    count = 0  
    for i in range(len(state)):  
        if state[i] != self.goal[i]:  
            count += 1  
    return count + dictionary[self.goal]
```

For A*:

Problem: WHY → ASK, frequency cost

Node reduction: $100 * (755-634)/755 = 16.026\%$

Problem: BEAR → DUCK, frequency cost

Node reduction: $100 * (720-302)/720 = 58.056\%$

Problem	Heuristic	Cost	Nodes Expanded
WHY → ASK	h_0	38.86	755

WHY→ASK	Proposed	38.86	634
BEAR→ DUCK	h_0	38.24	720
BEAR→DUCK	Proposed	38.24	302

The experimental results support using the proposed heuristic for these two frequency cost problems. In both cases, the proposed heuristic produced the same solution cost as h_0 , so the optimal solution cost was preserved. At the same time, it reduced the number of expanded nodes from 755 to 634 for WHY → ASK and from 720 to 302 for BEAR → DUCK.

The reduction was approximately 16.026% for WHY → ASK and 58.056% for BEAR → DUCK.

The theoretical proof shows that the heuristic is admissible, while the experiments show that it can reduce the search effort on these two test problems.

Note: I used ChatGPT (GPT-5.6 Luna) during this assignment to help me understand the supplied DOGCAT code and assignment concepts. I first asked ChatGPT to explain the given code so that I could understand how the implementation works. I then developed my own approach and heuristic ideas and discussed them with ChatGPT. After writing my code, I used ChatGPT to review and verify my implementation and reasoning. I also used Python and library documentation to understand the supplied code. I verified the code and reasoning by comparing them with the assignment instructions, running the required testing, checking the resulting solution costs and expanded node counts.

For Question 4, I used ChatGPT as a generative AI system for proposing and evaluating the heuristic. The exact prompt and relevant AI output are included in Question 4 as required by the assignment.